

SkillSwap Campus

Implementation Guide and Project Map

Purpose	A simple reference for understanding, running, extending, and deploying the full-stack project.
Architecture	Vanilla JavaScript frontend + Express/TypeScript backend + Prisma/PostgreSQL.
Current integrations	Supabase PostgreSQL, JWT authentication, and YouTube Data API course search.

Read this document in order for setup and deployment, or use the section headings as a project reference.

1. What the project does

SkillSwap Campus is a peer learning platform for college students. A student creates an account, lists skills they can teach and learn, discovers other registered members, sends a swap request, schedules a learning session, exchanges messages, and finds free video courses.

User journey

Step	Student action	System result
1	Sign up / log in	Backend issues access and refresh JWT tokens.
2	Complete profile and skills	Profile and teaching/learning skills are stored in PostgreSQL.
3	Explore people and matches	Real database members are shown; match scoring ranks complementary skills.
4	Send or accept a request	A SwapRequest is created and moves through requested, accepted, cancelled, or completed states.
5	Message and complete session	Messages are attached to the swap; completed sessions issue credits.
6	Use Courses	The backend searches YouTube and returns safe course metadata without exposing the API key.

What is real versus demo

Authentication, profiles, skills, swap requests, sessions, messages, matching, credits, and course search use the backend/database. Homepage cards and some community content are visual demo content. Community posts created in the current browser are stored in localStorage for local demonstration.

2. Project structure

Location	Role
index.html	Page shell, header, footer, and script/style loading.
style.css	Responsive visual design and reusable UI classes.
script.js	Hash router, page rendering, forms, modals, interactions, and API calls.
api.js	Frontend API client, token storage, refresh logic, and authenticated requests.
data.js	Public/demo student and community data.
backend/src/index.ts	Express server, API routes, health endpoint, and static frontend hosting.
backend/src/routes	Auth, users, skills, swaps, sessions, messages, matches, credits, reviews, and courses endpoints.
backend/prisma/schema.prisma	PostgreSQL database models and relations.
backend/prisma/seed.ts	Optional verified demo members for local swap testing.

How requests move through the app

Browser page -> api.js -> Express route -> middleware (JWT/validation) -> Prisma -> Supabase PostgreSQL -> JSON response -> UI refresh. Courses follow the same path, but the backend additionally calls the YouTube Data API with the private server key.

Important design decision

The frontend and API are served by one Express deployment. This avoids cross-origin frontend configuration in production: the page uses relative /api/v1 URLs, so the same domain serves both UI and backend.

3. Database and backend guide

Main database models

Model	Why it exists
User	Account details, verification status, profile, and JWT ownership.
Skill	Reusable named skill with a category.
UserSkillTeach / UserSkillLearn	Many-to-many mapping between a student and the skills they teach/learn.
SwapRequest	A request between two users, with a status and optional primary skill.
Session	Scheduled meeting attached to an accepted swap request.
Message	Conversation messages belonging to an accepted/scheduled/completed swap.
CreditTransaction	Immutable ledger entries, issued when a session is completed.
Review / Notification / RefreshToken	Feedback, user alerts, and secure refresh-session tracking.

Important API groups

Route group	Examples
/api/health	Health check used locally and by Render.
/api/v1/auth	register, login, refresh, verify-email, logout.
/api/v1/users	current profile, member listing, profile update, skills.
/api/v1/swaps and /sessions	create/accept/decline/cancel swaps; schedule or complete sessions.
/api/v1/messages	read/send messages for a valid swap.
/api/v1/courses	server-side YouTube search: /courses?q=Python.

4. Local setup and daily development

Prerequisites

Install Node.js 20+ and create a Supabase PostgreSQL project (or use another PostgreSQL database). The backend package file contains the required project dependencies.

Environment setup

Copy backend/.env.example to backend/.env. Keep .env only on your computer or deployment provider. It is excluded from Git by .gitignore.

Core variables

Variable	Purpose
DATABASE_URL	Runtime PostgreSQL connection.
DIRECT_URL	Direct/session PostgreSQL URL used by Prisma tooling.
JWT_ACCESS_SECRET / JWT_REFRESH_SECRET	Different long random values used to sign tokens.
YOUTUBE_API_KEY	Restricted YouTube Data API v3 key, used only by backend courses route.
FRONTEND_URL	Public URL used for links in emails; localhost locally, Render URL in production.

Run commands

```
cd "C:\everything\Skill swap-campus\backend" npm install npx prisma db push npm run db:seed npm run build npm start
```

Open <http://localhost:4000>. Keep the terminal open. Use Ctrl + C to stop the server. If port 4000 is occupied, stop the older Node process before starting again.

5. Testing checklist

Use two accounts when testing swaps. A user cannot send a swap request to themselves, and a real swap requires another registered database user.

Area	Check	Expected result
Server	Open /api/health	Returns status ok JSON.
Auth	Create account and log in	Tokens stored locally; header shows Profile and Log out.
Profile	Change name, college, bio	Saved values remain after page refresh.
Skills	Add teach and learn skills	Skills appear in My Skills and profile.
Explore	Search/filter other users	Database members appear after sign-in.
Swaps	Send, accept/decline/cancel request	Request state updates in Requests.
Messages	Message after an accepted swap	Message persists under that swap.
Courses	Search Python or Figma	Course cards load from YouTube without exposing the key.

Troubleshooting

Symptom	First action
EADDRINUSE port 4000	An older server is running. Stop it in its terminal with Ctrl + C, then run npm start.
Database table does not exist	Confirm DATABASE_URL/DIRECT_URL and run npx prisma db push locally.
Logged out unexpectedly	Log in again after a restart; verify backend is available and JWT secrets did not change.
No swap members	Create a second account or run npm run db:seed for development demo members.
Courses unavailable	Check YOUTUBE_API_KEY exists server-side and is restricted to YouTube Data API v3.

6. GitHub, deployment, and roadmap

GitHub safety

Commit source code, README.md, .env.example, package-lock.json, and Prisma schema. Never commit backend/.env, keys, database URLs with passwords, generated node_modules, or dist files.

Render deployment

Render setting	Value
Root Directory	backend
Build Command	npm install --include=dev && npm run build
Start Command	npm start
Health Check Path	/api/health
Pre-deploy command	Leave empty for the current project.

Render variable checklist

NODE_ENV=production; DATABASE_URL; DIRECT_URL; JWT_ACCESS_SECRET; JWT_REFRESH_SECRET; JWT_ACCESS_EXPIRES_IN=24h; JWT_REFRESH_EXPIRES_IN=7d; YOUTUBE_API_KEY; ALLOWED_EMAIL_DOMAINS; CORS_ORIGIN=*; FRONTEND_URL=your live Render URL; NODE_VERSION=22.13.1.

Recommended next improvements

Priority	Improvement	Why
High	Create formal Prisma migrations and tests	Safer schema updates and repeatable quality checks.
High	Production email verification and password reset	Completes account lifecycle outside development mode.
Medium	Real-time Socket.IO chat	Improves messaging beyond REST refresh.
Medium	Course bookmarks and learning progress	Connects YouTube resources to each learner profile.
Medium	Admin moderation and reporting UI	Uses existing report models for safer community management.

End of guide